

PlanO Operations Manual and Implementation Blueprint

Executive summary

PlanO is currently a **working geometry + selection system** that stops before it can reliably generate revenue. Based on your provided **PlanO FULL DEEP AUDIT** (treated as authoritative), the platform successfully completes the “hard” parts of the pipeline—**floor plan upload → raster processing → SceneJSON import → 2D/3D planning → 3D surface selection → service/material selection → ledger creation**—but is blocked at the money layer: **pricing, quote locking, payments, durable persistence, and authentication/tenant isolation**.

Three categories of issues prevent PlanO from operating as a secure, revenue-generating system:

- **Revenue blockers:** `03_invoice/` is empty → estimates are \$0; `04_stripe/` is empty → no Checkout; confirmation/assignment is stubbed.
- **Trust killers:** critical state (committed ledger, drafts, selections) disappears on refresh; missing door/window modeling can produce misleading quantities unless disclosed and bounded.
- **Security/business risk:** permissive `postMessage("*")` cross-window messaging, secrets committed to repo, and unauthenticated file upload endpoints create leakage and financial exposure. Cross-window messaging must restrict and validate origins per Mozilla ¹ guidance. ²

This document connects the missing dots into a **Phase 1 MVP that reaches end-to-end payment in test mode** while enforcing non-negotiable financial/security invariants and laying a clean path to production.

Non-negotiable invariants

These must be true before any production launch:

1. **Server-side quote locking:** the payable amount is computed and locked on the server; the client never supplies or edits payment totals.
2. **Webhook verification:** Stripe webhook signatures are verified using the payload + `Stripe-Signature` header + endpoint secret; unverified events are rejected. ³
3. **Idempotency:** payment session creation and webhook processing are idempotent (no double orders). Stripe idempotency keys must be used for POST requests. ⁴
4. **Tenant scoping:** every read/write is scoped by tenant (client/company) to prevent cross-tenant leakage. ⁵
5. **No secrets in repo:** rotate exposed keys immediately and implement real secrets management practices. ⁶
6. **Strict `postMessage` security:** never use `"*"` in production; enforce expected origin + message schema validation. ²

7. **Secure session handling**: prefer secure cookies (`HttpOnly` , `Secure` , `SameSite`) and CSRF defenses when cookies authenticate requests. 7

Architecture and data flows

Current architecture per audit

The audit describes three active components and two empty “money” modules:

- **PlanO UI** (`00_frontend_skeleton/plano-ui/`): orchestrates upload, planning, selection, and the committed ledger (`Planner.tsx` , `PlannerFrame.tsx` , `plannerState.tsx`).
- **React Planner Engine** (`02_react_planner/react-planner/`): iframe-hosted planner with a `postMessage` command protocol; emits scene JSON and selection events (`renderer.tsx` , `viewer3d.tsx`).
- **Raster pipeline** (`01_raster/`): FastAPI `POST /upload-plan` + subprocess pipeline calls `RasterScan` and converts to SceneJSON (`app.py` , `send_to_rasterscan.py` , `convert_raster_to_scene.py`).
- **Pricing** (`03_invoice/`): empty.
- **Payments** (`04_stripe/`): empty.

The pipeline status described by the audit is:

image → raster → SceneJSON → planner → service selection → pricing → payment → confirmation
✓✓✓✓ ~85% ✗ ✗ ✗

Target architecture for an operational MVP

The fastest safe path is to **extend the existing FastAPI service** (currently in `01_raster/server/app.py`) into a single backend that owns:

- authentication + authorization,
- project persistence,
- quote calculation + locking,
- Stripe session creation + webhook fulfillment.

This is a classic “thin client, authoritative server” pattern required for payments.

flowchart LR

```
subgraph Frontend["PlanO UI (00_frontend_skeleton/plano-ui)"]
    UIPlanner["Planner.tsx + plannerState.tsx"]
    UIFrame["PlannerFrame.tsx (iframe bridge)"]
    UIEstimate["Estimate.tsx / Quote / Payment / Confirmation"]
end
```

```
subgraph Engine["React Planner Iframe (02_react_planner/react-planner)"]
    Renderer["demo/src/renderer.tsx (postMessage protocol)"]
end
```

```

Viewer3D["src/components/viewer3d/viewer3d.tsx (surface selection)"]
end

subgraph Backend["FastAPI Backend (01_raster/server/app.py + new modules)"]
  Upload["/upload-plan (existing)"]
  Auth["/api/auth/* (new)"]
  Projects["/api/projects/* (new)"]
  Quotes["/api/quotes/* (new)"]
  Payments["/api/payments/* (new)"]
  Webhooks["/api/payments/webhook (new)"]
  DB[(Database)]
end

UIPlanner <--> UIFrame
UIFrame <--> Renderer
Renderer <--> Viewer3D
UIFrame --> Upload
UIEstimate --> Quotes
UIEstimate --> Payments
Webhooks --> DB
Auth --> DB
Projects --> DB
Quotes --> DB
Payments --> DB

```

MVP vs full platform scope

Capability	MVP needed to collect payments (Phase 1)	Full platform (post-MVP)
Auth	Email/password + session cookies + roles	MFA, social login, device/session management
Persistence	Projects + scene JSON + ledger + quotes + payments	Versioning, collaboration, professional portals
Pricing	Static catalog JSON per region/ company; server compute	Admin pricing UI, dynamic catalog per contractor, promotions
Payments	Stripe Checkout (test mode), webhooks, idempotency	Stripe Connect split payouts, refunds, disputes
Tenant model	Client + Company separation; scoped queries	Full multi-tenant isolation patterns (RLS, caching isolation) ⁵
Planner	Existing iframe + selection + ledger	Door/window modeling, object services, richer catalog

Ambiguities that require file verification

Your audit provides approximate line references and highlights some likely gating conditions. These should be verified directly in the following paths:

- `00_frontend_skeleton/plano-ui/src/pages/client/Planner.tsx`: confirm the exact condition that gates `handleCatalogServiceSelected` to painting only ("line ~82" per audit).
- `00_frontend_skeleton/plano-ui/src/components/planner/PlannerFrame.tsx`: confirm message origin handling and where `targetOrigin` is specified.
- `02_react_planner/react-planner/demo/src/renderer.tsx`, `02_react_planner/react-planner/src/components/viewer3d/viewer3d.tsx`, `02_react_planner/react-planner/src/components/catalog-view/catalog-item.tsx`: confirm every `postMessage(..., "*")` sender and every receiver that does not validate `event.origin`. MDN emphasizes specifying a strict `targetOrigin` and validating message origins. ²
- `01_raster/server/app.py`: confirm whether it already exposes routes beyond `/upload-plan`, and whether it is the preferred place to add `/api/*` endpoints (safe default: yes).

Security and leakage prevention

This section prioritizes guidance from OWASP ⁸, MDN, Stripe, and FastAPI.

Top risks with severity and mitigations

Risk	Severity	Why it matters	Mitigation (Phase 1)
<code>postMessage("*")</code> and missing <code>origin</code> checks	Critical	Enables cross-window data leakage and spoofed commands if any untrusted origin interacts with the channel. ²	Replace <code>*</code> with strict origins; verify <code>event.origin</code> + message schema on receivers. ²
Secrets committed to repo (<code>01_raster/.env</code> key)	Critical	Key compromise can incur direct costs and service abuse; violates secrets hygiene. ⁶	Rotate keys immediately; purge; add secrets scanning; move to environment/secret manager. ⁶
No server-side pricing / quote locking	Critical	Client can tamper totals; cannot safely charge money without authoritative totals.	Build quotes as server records; lock quote before payment; payment uses locked quote only (invariants).
No webhook verification	Critical	Attackers can forge "paid" callbacks if you trust client return pages. Stripe requires signature verification. ³	Implement webhook signature verification + idempotent event handling. ⁹

Risk	Severity	Why it matters	Mitigation (Phase 1)
Unauthenticated file uploads	High	File upload is a common attack surface; unrestricted uploads can lead to system takeover or DoS. ¹⁰	Require auth (when public), strict allowlist, size limits, store outside webroot, scan/sandbox. ¹¹
No tenant isolation / IDOR risk	High	Users/pros could access each other's projects/quotes if endpoints use raw IDs. ¹²	Tenant-scoped queries; non-guessable IDs; authorization gates per route. ¹²
No durable state	High	Users lose work on refresh → conversion and trust collapse.	Server persistence + autosave; local cache as backup. UX must show save status. ¹³

Required security changes with files, effort, and impact

Below is a Phase 1 “security change matrix.” Each item includes file paths from the audit plus safe defaults for new backend modules.

Change	Files to modify or add	Priority	Effort	Security impact
Restrict cross-window memory leaks: strict origin <code>postMessage</code>	<code>02_react_planner/.../viewer3d.tsx</code> , <code>02_react_planner/.../renderer.tsx</code> , <code>02_react_planner/.../catalog-item.tsx</code> , <code>00_frontend_skeleton/.../PlannerFrame.tsx</code>	P0	3–6h	Prevents spoofing/data leakage via <code>postMessage</code> . ²
Rotate and remove committed API keys	<code>01_raster/.env</code> (remove from repo), deployment env	P0	1–2h	Stops immediate financial exposure; aligns w/ secrets best practices. ⁶
Secure auth + session strategy	Add backend auth modules; update <code>ClientLayout.tsx</code> , <code>App.tsx</code> routes	P0	10–16h	Prevents account takeover and unauthorized access; enables tenant isolation. ¹⁴

Change	Files to modify or add	Priority	Effort	Security impact
File upload hardening	<code>01_raster/server/app.py</code> upload handler	P0	4–8h	Reduces high-impact upload vulnerabilities. ¹⁵
Security logging & audit trails	Add backend logging layer	P1	4–8h	Enables detection and incident response. ¹⁶

Key security guidelines you should explicitly adopt

- **postMessage security**: specify an exact `targetOrigin` and validate received message origins. ²
- **Password storage**: use a modern password hashing scheme; OWASP recommends Argon2id with baseline parameters. ¹⁷
- **Session cookies**: use `HttpOnly`, `Secure`, and `SameSite` appropriately. ¹⁸
- **Uploads**: store untrusted files outside webroot; restrict permissions; scan where feasible. ¹¹
- **Secrets management**: centralize, audit, rotate, and avoid embedding secrets in code. ⁶
- **Payments**: verify Stripe webhooks; do not rely on a redirect page for payment truth. ¹⁹

Database schema and tenant isolation

The audit states “nothing exists” for database/auth. Therefore, the Phase 1 blueprint includes a **minimum viable schema** that supports: users, companies, projects, scene/ledger persistence, quotes, payments, and webhook idempotency.

Tenant model

PlanO is inherently multi-tenant (clients vs professionals/companies). OWASP notes multi-tenant apps must enforce tenant isolation to prevent cross-tenant attacks. ⁵

Safe default model:

- A **client user** owns projects.
- A **company** owns pricing catalogs.
- A **quote** is computed either against a default catalog (MVP) or a company catalog (later).
- Every query enforces tenant-scoped filters (never `WHERE id = :id` alone).

Minimum viable schema table

The following table is sized for MVP correctness and revenue safety. Replace `TEXT` UUIDs with specific UUID types where your ORM supports it.

Table	Columns (type)	Indexes	Constraints
users	id TEXT PK, email TEXT, password_hash TEXT, role TEXT, created_at TIMESTAMP, last_login_at TIMESTAMP	UNIQUE(email)	role IN ('client','pro
companies	id TEXT PK, name TEXT, region TEXT, status TEXT, created_at TIMESTAMP	INDEX(region), INDEX(status)	status IN ('pending','
company_users	company_id TEXT, user_id TEXT, company_role TEXT	UNIQUE(company_id,user_id)	FK to companies/users
projects	id TEXT PK, client_user_id TEXT, region TEXT, status TEXT, created_at TIMESTAMP, updated_at TIMESTAMP	INDEX(client_user_id), INDEX(status)	FK client_user_id → use
project_scene	project_id TEXT PK, scene_json JSON, version INT, updated_at TIMESTAMP	—	FK project_id → project
project_ledger	project_id TEXT PK, ledger_json JSON, version INT, updated_at TIMESTAMP	—	FK project_id → project
pricing_catalogs	id TEXT PK, scope TEXT, company_id TEXT NULL, region TEXT, currency TEXT, catalog_json JSON, version INT, effective_from TIMESTAMP	INDEX(company_id,region), INDEX(scope)	scope IN ('global','co

Table	Columns (type)	Indexes	Constraints
quotes	id TEXT PK, project_id TEXT, catalog_id TEXT, currency TEXT, subtotal_cents INT, fees_cents INT, total_cents INT, status TEXT, locked_at TIMESTAMP NULL, created_at TIMESTAMP	INDEX(project_id), INDEX(status)	status IN ('draft','calculated',
quote_line_items	id TEXT PK, quote_id TEXT, service TEXT, subservice TEXT, qty DECIMAL, unit TEXT, unit_price_cents INT, amount_cents INT, meta JSON	INDEX(quote_id)	FK quote_id → quotes(id)
payments	id TEXT PK, quote_id TEXT UNIQUE, provider TEXT, checkout_session_id TEXT, payment_intent_id TEXT NULL, status TEXT, created_at TIMESTAMP, updated_at TIMESTAMP	UNIQUE(checkout_session_id)	FK quote; status enum
webhook_events	id TEXT PK, provider TEXT, event_id TEXT, type TEXT, received_at TIMESTAMP, processed_at TIMESTAMP NULL, payload_hash TEXT	UNIQUE(provider,event_id)	Idempotency gate

Notes / assumptions: - JSON types depend on DB choice; Postgres supports JSONB; SQLite will accept text JSON for early MVP. - Quote totals are stored in integer cents to avoid float errors.

Pricing, quote lifecycle, and payments

Quote lifecycle model

PlanO must treat quotes as “financial artifacts” with immutable locking before payment.

```
stateDiagram-v2
    [*] --> Draft
    Draft --> Calculated: server calculates from ledger + catalog
    Calculated --> Locked: server locks totals and catalog version
    Locked --> Paid: Stripe webhook confirms payment
    Locked --> Expired: TTL reached or catalog changed
    Paid --> Assigned: (post-MVP) professional assignment
    Paid --> Closed: job completed / archived
    Draft --> Void: user cancels
    Calculated --> Draft: user edits ledger
```

Critical rule: only **Locked** quotes can proceed to payment.

Pricing engine requirements (what `03_invoice` must do)

Per audit, the committed ledger includes: - service + subservice, - material metadata (key/label/color/texture/dimensions), - surfaces map with per-surface geometry and total m².

Therefore pricing must: - map each committed ledger entry → one or more line items, - fetch applicable catalog by region/company scope, - compute totals and fees (platform commission) deterministically, - return a “Calculated” quote preview and support “Lock quote”.

Implementation location (safe default): - Add a `03_invoice/` Python module (or TypeScript module) and import it into FastAPI. The audit indicates `03_invoice/` currently exists but is empty, so creating `03_invoice/pricing.py` (or `calculate.ts`) matches your plan.

Pricing must be server-side because client totals are tamperable.

Stripe Checkout integration (test mode MVP)

Stripe’s current guidance for Checkout is: **create a Checkout Session server-side, then redirect the customer to the URL returned in the response.** ²⁰

Also note that Stripe has removed support for `stripe.redirectToCheckout` in Stripe.js in at least some tracks of the documentation; safest modern flow is redirecting directly to `session.url`. ²¹

Webhook verification and fulfillment

Stripe requires verifying webhook signatures using the raw request payload, the `Stripe-Signature` header, and your endpoint secret. ³

Idempotency is required both for Stripe API calls and webhook processing: - Stripe idempotency keys allow safe retries of POST operations. ⁴

- Your webhook handler must be idempotent by storing `event.id` and refusing to process duplicates.

PCI scope note

Using Stripe-hosted payment collection (Checkout) can reduce PCI burden by avoiding direct handling of sensitive card data in your servers. ²²

API endpoints and contracts

This section defines minimal endpoint surfaces required to make the platform operate end-to-end in test mode.

Backend endpoints (FastAPI) for Phase 1

Assumption: you will extend `01_raster/server/app.py` with `/api/*` routes. This aligns with FastAPI's supported security patterns (OAuth2 + JWT + password hashing) as a baseline. ²³

Auth endpoints

- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/logout`
- `GET /api/auth/me`

Auth implementation options (safe defaults): - Either (A) OAuth2 bearer JWT per FastAPI tutorial, or (B) JWT in secure cookies with CSRF tokens for state-changing ops. FastAPI's official tutorial demonstrates JWT + hashing patterns. ²⁴

- If cookie-based, apply OWASP cookie protections. ⁷

```
# pseudocode signatures (not full code)
POST /api/auth/register { email, password } -> { userId }
POST /api/auth/login    { email, password } -> Set-Cookie: session=...
GET  /api/auth/me       -> { userId, role, companyId? }
```

Project persistence endpoints

- `POST /api/projects` → create a project shell (region, address, service type)
- `PUT /api/projects/{id}/scene` → store scene JSON
- `PUT /api/projects/{id}/ledger` → store committed ledger

- `GET /api/projects/{id}` → retrieve project + scene + ledger + last quote

Tenant rule: only the owner (client) or permitted company roles can access.

Quote endpoints

- `POST /api/quotes/calculate` (returns calculated preview; status=calculated)
- `POST /api/quotes/{quoteId}/lock` (locks totals; returns locked quote)

```
POST /api/quotes/calculate
{ projectId } # server loads ledger/scene, applies catalog
-> { quoteId, status:"calculated", lineItems[], totals }

POST /api/quotes/{quoteId}/lock
{}
-> { quoteId, status:"locked", lockedAt, totals }
```

Payment endpoints

- `POST /api/payments/create-checkout-session` (takes locked quoteId; returns session URL)
- `POST /api/payments/webhook` (Stripe events; signature verified)

Stripe requires secret keys remain server-side only. ²⁵

Stripe idempotency usage: - Use `Idempotency-Key: quoteId` (or derived UUID) for Checkout Session creation. ⁴

postMessage contract hardening

Files cited by audit: - `02_react_planner/react-planner/demo/src/renderer.tsx` -
`02_react_planner/react-planner/src/components/viewer3d/viewer3d.tsx` -
`00_frontend_skeleton/plano-ui/src/components/planner/PlannerFrame.tsx`

MDN security guidance implies: - The sender should set a strict `targetOrigin`. - The receiver must validate `event.origin`. ²

Pseudocode receiver gate (engine side):

```
// renderer.tsx (pseudocode)
window.addEventListener("message", (event) => {
  if (event.origin !== EXPECTED_UI_ORIGIN) return;
  if (!isValidPlanoMsg(event.data)) return;
  handleCommand(event.data);
});
```

UX and operational procedures

UX principles to protect conversion and trust

Your audit highlights the biggest UX risk: users lose work and estimates can be misleading.

Core usability heuristics to adopt: - Visibility of system status: always show “uploading / processing / saved / pending payment confirmation.” ¹³

- Help users recover from errors: errors must be actionable and respect user effort. ²⁶

- User control and freedom: undo/remove committed services and provide safe exit points. ²⁷

Autosave and recovery patterns

Minimum UX requirement (Phase 1): - Autosave on every meaningful event: - after raster import complete (scene saved), - after each committed ledger update, - before user leaves planner page. - Display status: - “Saved just now” / “Saving...” / “Offline—changes saved locally” (if you implement local fallback).

Implementation blueprint using audited files: - `plannerState.tsx`: add persistence hooks that write to backend and maintain local fallback. - `Planner.tsx`: call autosave when `committedLedger` changes and when scene JSON changes.

Copy suggestions (doors/windows not modeled): When doors/windows are not converted to holes (audit: not implemented), wall areas may be overestimated. You should disclose this explicitly:

- Banner on estimate page:
- “Estimate is provisional: doors and windows aren’t automatically deducted yet. We’ll confirm measurements during final verification.”
- Tooltip near wall-area line items:
- “Wall area is calculated from modeled surfaces; openings may be included.”

This reduces user surprise and protects credibility.

Operational runbook basics

Secrets operations - Rotate all keys periodically; Stripe recommends key rotation with a contingency plan if exposed. ²⁸

- Centralize and audit secrets access (OWASP secrets management). ²⁹

Logging - Implement security logging appropriate for incident detection (OWASP logging cheat sheet). ¹⁶

- Never log credentials or tokens (practical hard rule; enforce via log filters).

Stripe operations - Use Stripe CLI for webhook forwarding in local dev; it provides the signing secret and can forward events. ³⁰

Prioritized roadmap, testing, and monitoring

Change list with file paths, pseudocode, effort, priority, security impact

This list is tailored to the audit's actual paths.

Fix routing and unblock the journey

- **Change:** GeoSelect routes to `/planner/2d` and `/planner/3d` (dead stubs); must route to `/planner`.
- **File:** `00_frontend_skeleton/plano-ui/src/pages/client/GeoSelect.tsx`
- **Priority:** P0
- **Effort:** 0.5h
- **Security impact:** Low (UX unblock)
- **Snippet:**

```
navigate("/planner"); // replace /planner/2d and /planner/3d
```

Persist critical state durably

- **Change:** committed ledger + drafts lost on reload; must persist to DB (and optionally local fallback).
- **Files:**
- `00_frontend_skeleton/plano-ui/src/state/plannerState.tsx`
- `00_frontend_skeleton/plano-ui/src/pages/client/Planner.tsx`
- Backend additions under `01_raster/server/`
- **Priority:** P0
- **Effort:** 8–14h
- **Security impact:** Medium (prevents data loss; enables auditability)

Pseudocode (frontend):

```
useEffect(() => {  
  debounce(() => api.saveLedger(projectId, serializeLedger(committedLedger)),  
    500);  
}, [committedLedger]);
```

Harden postMessage channel

- **Change:** replace `targetOrigin=""` and validate `event.origin`.
- **Files:**
- `02_react_planner/.../viewer3d.tsx`
- `02_react_planner/.../renderer.tsx`
- `02_react_planner/.../catalog-item.tsx`
- `00_frontend_skeleton/.../PlannerFrame.tsx`
- **Priority:** P0

- **Effort:** 3–6h
- **Security impact:** Critical (prevents spoofing & leakage) ²

Implement pricing engine in 03_invoice

- **Change:** Estimate.tsx uses rate=0; build catalog + compute line items on server.
- **Files:**
 - Create: 03_invoice/catalogs/default.json
 - Create: 03_invoice/pricing.py (or calculate.ts)
 - Update: 00_frontend_skeleton/plano-ui/src/pages/client/Estimate.tsx
 - Add endpoint: 01_raster/server/app.py → POST /api/quotes/calculate
- **Priority:** P0
- **Effort:** 10–18h
- **Security impact:** Critical (money correctness)

Implement Stripe payments in 04_stripe + backend

- **Change:** empty 04_stripe/; add Checkout session creation and webhook verification.
- **Files:**
 - Create: 04_stripe/stripe_service.py (or integrate directly in FastAPI)
 - Update: 00_frontend_skeleton/plano-ui/src/pages/client/Payment.tsx
 - Update: 00_frontend_skeleton/plano-ui/src/pages/client/Confirmation.tsx
 - Add endpoints: 01_raster/server/app.py
- **Priority:** P0
- **Effort:** 12–20h
- **Security impact:** Critical (prevents forged payments) ⁹

Phase 1 implementation checklist to reach end-to-end payment in test mode

This is a developer-executable sequence from the audited baseline to “paid quote” in test mode.

Backend setup

Packages to add (Python)

```
# from 01_raster/ (assumed)
pip install stripe
pip install sqlalchemy alembic psycopg2-binary # if Postgres
pip install argon2-cffi
pip install python-jose[cryptography] # if using JWT
```

Stripe webhooks require signature verification using the endpoint secret. ³

Environment variables (do not commit) - STRIPE_SECRET_KEY (server-side only) ²⁵
 - STRIPE_WEBHOOK_SECRET (whsec_...) ³¹

- DATABASE_URL
- Rotate the RasterScan key and remove it from git history (secrets management best practices). 6

Minimal FastAPI endpoint signatures (in 01_raster/server/app.py)

```
# Pseudocode only
@app.post("/api/quotes/calculate")
def calculate_quote(project_id: str, user=Depends(require_auth)):
    ...

@app.post("/api/quotes/{quote_id}/lock")
def lock_quote(quote_id: str, user=Depends(require_auth)):
    ...

@app.post("/api/payments/create-checkout-session")
def create_checkout_session(quote_id: str, user=Depends(require_auth)):
    # Stripe Checkout Session create with idempotency key
    ...

@app.post("/api/payments/webhook")
async def stripe_webhook(request: Request):
    # verify Stripe-Signature using endpoint secret
    ...
```

Stripe idempotency guidance: use idempotency keys for POST requests to safely retry. 4

Webhook local testing

```
stripe listen --forward-to localhost:4242/api/payments/webhook
```

Stripe CLI can forward events and provides the webhook signing secret for verification. 30

Frontend setup

Packages to add (UI)

```
cd 00_frontend_skeleton/plano-ui
npm install @stripe/stripe-js
```

Note: modern Stripe guidance for Checkout commonly redirects using the session URL, and Stripe has removed `stripe.redirectToCheckout` in at least some tracks—so prefer `window.location = session.url`. 32

Update routes (audit says Quote/Payment/Confirmation are not routed) - `00_frontend_skeleton/plano-ui/src/App.tsx`: add routes for `/quote`, `/payment`, `/confirmation`.

- `Landing.tsx`: remove or update dead routes `/upload` and `/create` (requires file verification; audit says they don't exist).

Payment flow (recommended) - Client requests `/api/payments/create-checkout-session` with `quoteId`. - Backend returns `{ url }`. - Frontend redirects `window.location.href = url`.

Stripe documents Checkout Sessions provide a URL that redirects customers to a Stripe-hosted payment page. ³³

Testing and monitoring

Tests to write

Pricing correctness - Unit tests for `03_invoice/pricing.py`: - deterministic totals from a fixed ledger input, - rounding rules, - catalog selection by region/company, - doors/windows disclaimer gates (if modeled later).

Auth - Login/register tests: - password hashing validation (OWASP recommended algorithms). ¹⁷
- session cookie flags: `HttpOnly`, `Secure`, `SameSite`. ¹⁸

Tenant isolation - Integration tests: - user A cannot fetch user B's project/quote by id (OWASP multi-tenant isolation principles). ⁵

Webhook processing - Signature verification test with known payload + secret (Stripe webhook verification). ³
- Idempotency test: - same webhook `event.id` delivered twice results in one state change.

Monitoring and alerting (MVP set)

Use OWASP logging guidance for security-relevant event visibility. ¹⁶

Track and alert on: - Upload failures (`/upload-plan` status codes, processing timeouts).
- RasterScan API usage spikes (protects budget; rotate keys on anomaly). ³⁴
- Quote calculation failures and outlier totals (potential pricing bug).
- Checkout Session creation failures (Stripe API errors).
- Webhook signature verification failures (could indicate misconfiguration or attack). ³⁵
- Payment mismatch attempts (client tries to pay an unlocked quote).

Top 10 immediate tasks

1. **Rotate and revoke exposed secrets**, remove `.env` credentials from git history, and enforce secrets management discipline (no secrets in repo). ⁶

2. **Fix `postMessage` security:** replace `targetOrigin=""` with strict origins and verify `event.origin` + schema in all receivers (`viewer3d.tsx` , `renderer.tsx` , `catalog-item.tsx` , `PlannerFrame.tsx`). ²
3. **Fix `GeoSelect` routing** to `/planner` (`GeoSelect.tsx`) to unblock the main journey immediately.
4. **Introduce a database + migrations** (safe default: Postgres in prod, SQLite allowed for dev) and implement the minimum schema (users/projects/quotes/payments/webhook_events).
5. **Implement authentication and roles** (register/login/me/logout) with OWASP-aligned password hashing and secure session cookies + CSRF strategy. ³⁶
6. **Build `03_invoice` pricing engine** and `/api/quotes/calculate` + `/api/quotes/{id}/lock` endpoints; update `Estimate.tsx` to display server-computed line items instead of `rate=0`.
7. **Implement Stripe Checkout Session creation** (`/api/payments/create-checkout-session`) using locked quotes, Stripe secret key server-side only, and Stripe idempotency keys for POST. ³⁷
8. **Implement Stripe webhook endpoint** with signature verification and idempotent event processing (`/api/payments/webhook`), tested via Stripe CLI. ³⁸
9. **Add durable autosave:** persist scene JSON + committed ledger server-side on changes and show "Saving/Saved" UI states (`Planner.tsx` , `plannerState.tsx`). ¹³
10. **Add UX disclosures and error recovery:** provisional estimate disclaimers (doors/windows not deducted yet), user-friendly error messages, and safe recovery paths for failed uploads or payment pending. ³⁹

¹ ⁶ ²⁹ Secrets Management - OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html?utm_source=chatgpt.com

² Window: `postMessage()` method - Web APIs - MDN Web Docs

https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage?utm_source=chatgpt.com

³ ⁹ ¹⁹ ³⁸ Receive Stripe events in your webhook endpoint

https://docs.stripe.com/webhooks?utm_source=chatgpt.com

⁴ Idempotent requests | Stripe API Reference

https://docs.stripe.com/api/idempotent_requests?utm_source=chatgpt.com

⁵ ¹² Multi Tenant Security - OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html?utm_source=chatgpt.com

⁷ ¹⁸ Session Management - OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html?utm_source=chatgpt.com

⁸ ¹⁴ ¹⁷ ³⁶ Password Storage - OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html?utm_source=chatgpt.com

¹⁰ Unrestricted File Upload

https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload?utm_source=chatgpt.com

¹¹ ¹⁵ File Upload - OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html?utm_source=chatgpt.com

¹³ 10 Usability Heuristics for User Interface Design

https://www.nngroup.com/articles/ten-usability-heuristics/?utm_source=chatgpt.com

16 **Logging - OWASP Cheat Sheet Series**

https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html?utm_source=chatgpt.com

20 32 **Accept a payment**

https://docs.stripe.com/payments/accept-a-payment?utm_source=chatgpt.com

21 **Removes support for the redirectToCheckout method**

https://docs.stripe.com/changelog/clover/2025-09-30/remove-redirect-to-checkout?utm_source=chatgpt.com

22 **What is PCI DSS compliance?**

https://stripe.com/guides/pci-compliance?utm_source=chatgpt.com

23 24 **OAuth2 with Password (and hashing), Bearer with JWT ...**

https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/?utm_source=chatgpt.com

25 37 **API keys - Stripe Documentation**

https://docs.stripe.com/keys?utm_source=chatgpt.com

26 39 **Error-Message Guidelines - NN/G**

https://www.nngroup.com/articles/error-message-guidelines/?utm_source=chatgpt.com

27 **User Control and Freedom (Usability Heuristic #3)**

https://www.nngroup.com/articles/user-control-and-freedom/?utm_source=chatgpt.com

28 34 **Best practices for managing secret API keys**

https://docs.stripe.com/keys-best-practices?utm_source=chatgpt.com

30 **Use the Stripe CLI**

https://docs.stripe.com/stripe-cli/use-cli?utm_source=chatgpt.com

31 35 **Resolve webhook signature verification errors**

https://docs.stripe.com/webhooks/signature?utm_source=chatgpt.com

33 **How Checkout works**

https://docs.stripe.com/payments/checkout/how-checkout-works?utm_source=chatgpt.com